

Do Solve Denso ao BiCGSTAB Reescalonado: Um Solver Linear Esparso para o Sistema de Sela de Navier–Stokes em GPU

Projeto `femns`

Julho de 2026

Resumo

Este documento registra a substituição do solver linear denso do pacote `femns` (`torch.linalg.solve` sobre a matriz convertida para densa a cada passo de tempo) por um solver iterativo esparsa – BiCGSTAB, implementado em PyTorch puro – e explica os conceitos algébricos por trás dessa mudança: subespaços de Krylov, a derivação do BiCGSTAB a partir do BiCG/CGS, o mau condicionamento do sistema de sela (*saddle-point*) de Navier–Stokes e a técnica de reescalamento (equilibração) que o torna solúvel por esse método.

Sumário

1	Contexto e motivação	1
2	O sistema linear de Navier–Stokes discretizado	1
3	Métodos de subespaço de Krylov	2
4	Do BiCG ao BiCGSTAB	2
4.1	BiCG: o problema com $A^\top$	2
4.2	CGS: elimina $A^\top$, mas amplifica ruído	3
4.3	BiCGSTAB: estabiliza o quadrado	3
5	O algoritmo BiCGSTAB (com pré-condicionamento)	3
5.1	Por que não usamos o pré-condicionador de Jacobi	4
6	O sistema de sela é mal condicionado: reescalamento de pressão	5
6.1	O diagnóstico	5
6.2	A técnica: reescalamento diagonal (equilibração)	5
6.3	Resultado	6
7	Warm-start entre passos de tempo	6
8	Resumo das mudanças no código	6
9	Validação de desempenho e correção	7
10	Conclusão e limitações	7

1 Contexto e motivação

O solver do `femns` monta, a cada passo de tempo, um sistema linear esparso $Ax = b$ vindo da discretização de Galerkin (elemento MINI) das equações de Navier–Stokes incompressíveis com avanço temporal por Euler implícito. Até a versão anterior, esse sistema era resolvido convertendo A para uma matriz densa (`A.to_dense()`) e chamando `torch.linalg.solve`. Isso funciona, mas tem custo $O(n^3)$ no tamanho do sistema n e desperdiça toda a esparsidade da matriz – na malha de teste (Poiseuille, $n \approx 19,000$ incógnitas) o solve denso levava $\approx 6,5$ s por passo de tempo, mesmo rodando na GPU.

A motivação imediata para mudar isso foi rodar o projeto em uma GPU AMD (Radeon RX 6750 XT, via ROCm). Ao investigar solvers esparsos nativos do PyTorch, encontramos que `torch.sparse.spsolve` existe, mas falha explicitamente em builds ROCm:

```
RuntimeError: Calling linear solver with sparse tensors requires compiling PyTorch
with CUDA cudSS and is not supported in ROCm build.
```

Ou seja, a única via de solver esparso “de fábrica” do PyTorch é exclusiva de CUDA/NVIDIA (usa a biblioteca cuDSS por baixo). Como alternativa portátil – que funciona igual em CUDA, ROCm ou CPU –, implementamos um método iterativo de Krylov (BiCGSTAB) inteiramente com as operações de tensor esparso que o PyTorch já suporta em qualquer *backend*: produto matriz-vetor esparso (`torch.mm`) e álgebra densa de vetor.

2 O sistema linear de Navier–Stokes discretizado

Antes de discutir o solver, vale relembrar a estrutura algébrica do sistema que ele precisa resolver, porque é essa estrutura que dita quais métodos se aplicam.

A cada passo de tempo, com o termo advectivo linearizado na velocidade do passo anterior, o `femns` monta o sistema em blocos

$$Ax = \underbrace{\begin{pmatrix} \frac{1}{\Delta t}M + \frac{1}{Re}K & 0 & -G_x \\ 0 & \frac{1}{\Delta t}M + \frac{1}{Re}K & -G_y \\ D_x & D_y & 0 \end{pmatrix}}_A \begin{pmatrix} v_x \\ v_y \\ p \end{pmatrix} = \begin{pmatrix} b_{v_x} \\ b_{v_y} \\ b_p \end{pmatrix} = b, \quad (1)$$

onde M é a matriz de massa, K a de rigidez (laplaciano) do elemento MINI, G_x, G_y os operadores de gradiente discreto e $D_x = -G_x^\top$, $D_y = -G_y^\top$ o operador de divergência (restrição de incompressibilidade). Chamando $A_{vv} = \frac{1}{\Delta t}M + \frac{1}{Re}K$ o bloco de velocidade, note que:

- **A é simétrica:** o bloco (p, v_x) é $D_x = -G_x^\top$, exatamente a transposta do bloco $(v_x, p) = -G_x$ – o mesmo vale para v_y . Isso é a estrutura clássica de um sistema de *sela* (*saddle-point*) proveniente de uma formulação mista (velocidade–pressão).
- **A não é definida positiva:** o bloco (p, p) é identicamente nulo (a equação de continuidade não tem termo de pressão), então A tem autovalores positivos e negativos – é simétrica *indefinida*.

Essa segunda propriedade é decisiva: o método do Gradiente Conjugado (CG), a escolha natural para sistemas simétricos, exige que a matriz seja **definida positiva** para garantir convergência (e para que as quantidades computadas, como o passo ótimo, sejam bem definidas). Como A aqui é indefinida, CG não se aplica diretamente – seria preciso MINRES (que lida com simétrica indefinida) ou um método para matrizes não-simétricas em geral, como o **BiCGSTAB**, que foi a escolha feita aqui por ser mais amplamente aplicável (não assume nem exige simetria) e por ter uma implementação direta em termos de produtos matriz-vetor.

3 Métodos de subespaço de Krylov

Métodos de Krylov resolvem $Ax = b$ construindo, a partir de um chute inicial x_0 e do resíduo $r_0 = b - Ax_0$, uma sequência de aproximações x_m tais que o erro $x_m - x_0$ pertence ao *subespaço de Krylov*

$$\mathcal{K}_m(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{m-1}r_0\}. \quad (2)$$

Equivalentemente, o resíduo $r_m = b - Ax_m$ pode sempre ser escrito como $r_m = \varphi_m(A)r_0$ para algum polinômio φ_m de grau m com $\varphi_m(0) = 1$. Diferentes métodos de Krylov correspondem a diferentes critérios para escolher esse polinômio:

- **CG**: escolhe φ_m que minimiza $\|r_m\|_{A^{-1}}$ (exige A simétrica definida positiva);
- **GMRES**: escolhe φ_m que minimiza $\|r_m\|_2$ diretamente (aplica-se a qualquer A não-singular, mas o custo de memória cresce com m , pois precisa guardar todos os vetores de Krylov anteriores);
- **BiCG** e seus descendentes (**CGS**, **BiCGSTAB**): escolhem φ_m por uma relação de recorrência de custo constante por iteração (não crescente com m), à custa de não garantir minimização monotônica do resíduo – é a família usada aqui.

4 Do BiCG ao BiCGSTAB

4.1 BiCG: o problema com $A^\top$

O **Gradiente Bi-Conjugado** (BiCG, Fletcher 1976) generaliza o CG para matrizes não-simétricas introduzindo um sistema “sombra” $A^\top x^* = b^*$ e um resíduo sombra arbitrário $\hat{r}_0$ (tipicamente $\hat{r}_0 = r_0$). O método gera duas sequências biortogonais, $r_m = \varphi_m(A)r_0$ e $\hat{r}_m = \varphi_m(A^\top)\hat{r}_0$, tais que $(\hat{r}_i, r_j) = 0$ para $i \neq j$. Isso funciona, mas tem duas desvantagens práticas: (i) exige um produto matriz-vetor com $A^\top$ a cada iteração, dobrando o custo (e sendo inconveniente quando só se tem acesso a A via sua ação, não sua transposta explícita); (ii) a convergência do resíduo costuma ser irregular, com oscilações grandes.

4.2 CGS: elimina $A^\top$, mas amplifica ruído

O **Gradiente Conjugado ao Quadrado** (CGS, Sonneveld 1989) observa que, como as quantidades do BiCG só entram em produtos internos que envolvem $\varphi_m(A^\top)\hat{r}_0$ contra outros vetores, é possível reescrever a recorrência de forma que o resíduo efetivo passe a ser $r_m = \varphi_m(A)^2 r_0$ – ou seja, o mesmo polinômio *aplicado duas vezes*, usando só A (nunca $A^\top$). Isso resolve o problema (i), mas piora o (ii): elevar φ_m ao quadrado também eleva ao quadrado qualquer componente de crescimento que o polinômio tenha, tornando a convergência do CGS tipicamente *mais* errática que a do BiCG, com picos de resíduo grandes no meio do processo (podendo até causar problemas numéricos em precisão finita).

4.3 BiCGSTAB: estabiliza o quadrado

O **BiCGSTAB** (van der Vorst, 1992) resolve o problema (ii) do CGS substituindo o *segundo* φ_m (o que o CGS aplica de novo, gerando o quadrado) por uma sequência de polinômios de grau 1 ψ_m , escolhidos localmente a cada iteração para minimizar a norma do resíduo intermediário – uma espécie de passo de GMRES(1)/mínimos quadrados local, intercalado com a recorrência do tipo BiCG. O resíduo final é

$$r_m = \psi_m(A) \varphi_m(A) r_0, \quad (3)$$

onde $\psi_m(A) = \prod_{j=1}^m (I - \omega_j A)$ é construído incrementalmente: a cada iteração j , depois de obter um resíduo intermediário s (análogo ao que o CGS usaria diretamente), calcula-se

$$\omega_j = \frac{(As, s)}{(As, As)}, \quad (4)$$

que é exatamente o escalar que minimiza $\|s - \omega As\|_2$ sobre $\omega \in \mathbb{R}$ – um passo de descida ótimo de grau 1. Isso mantém o custo por iteração igual ao do CGS (dois produtos matriz-vetor com A , nunca com $A^\top$), mas com convergência muito mais suave e estável na prática, sendo hoje o método padrão de facto para sistemas não-simétricos (ou simétricos indefinidos, como o nosso) quando GMRES completo é caro demais em memória.

5 O algoritmo BiCGSTAB (com pré-condicionamento)

A seguir, o BiCGSTAB pré-condicionado à direita, na forma implementada em `femns.krylov.bicgstab` (numeração de linha entre colchetes). O pré-condicionador K^{-1} é aplicado às direções de busca p, s antes de multiplicá-las por A – se $K = I$ (identidade), recupera-se o BiCGSTAB sem pré-condicionamento.

Entrada: A, b , chute inicial x_0 , tolerância tol , pré-condicionador $K^{-1}(\cdot)$.

```
[1]   $r_0 \leftarrow b - Ax_0; \quad \hat{r}_0 \leftarrow r_0$  (arbitrário)
[2]   $\rho_0 \leftarrow \alpha \leftarrow \omega_0 \leftarrow 1; \quad v_0 \leftarrow p_0 \leftarrow 0$ 
[3]  para  $j = 1, 2, 3, \dots$  faça
[4]     $\rho_j \leftarrow (\hat{r}_0, r_{j-1})$ 
[5]     $\beta \leftarrow (\rho_j / \rho_{j-1}) (\alpha / \omega_{j-1})$ 
[6]     $p_j \leftarrow r_{j-1} + \beta (p_{j-1} - \omega_{j-1} v_{j-1})$ 
[7]     $y \leftarrow K^{-1} p_j$  (pré-condiciona a direção de busca)
[8]     $v_j \leftarrow Ay$ 
[9]     $\alpha \leftarrow \rho_j / (\hat{r}_0, v_j)$ 
[10]    $s \leftarrow r_{j-1} - \alpha v_j$ 
[11]   se  $\|s\| / \|b\| \leq \text{tol}$  :  $x_j \leftarrow x_{j-1} + \alpha y$ ; pare
[12]    $z \leftarrow K^{-1} s$  (pré-condiciona o resíduo intermediário)
[13]    $t \leftarrow Az$ 
[14]    $\omega_j \leftarrow (t, s) / (t, t)$  (Eq. (4): passo de descida ótimo)
[15]    $x_j \leftarrow x_{j-1} + \alpha y + \omega_j z$ 
[16]    $r_j \leftarrow s - \omega_j t$ 
[17]   se  $\|r_j\| / \|b\| \leq \text{tol}$  : pare
```

Cada iteração custa exatamente **dois** produtos matriz-vetor esparsos (Ay e Az) mais um punhado de produtos internos e combinações lineares de vetores densos – todas operações que já confirmamos funcionar sem ressalvas na build ROCm do PyTorch. É por isso que o método é atrativo aqui: não precisamos de nenhuma biblioteca de fatoração esparsa específica do fabricante da GPU.

5.1 Por que não usamos o pré-condicionador de Jacobi

A escolha mais simples de K^{-1} é o pré-condicionador de **Jacobi** (diagonal): $K^{-1}v = D^{-1}v$, onde $D = \text{diag}(A)$. Implementamos essa opção (`jacobi_preconditioner` em `krylov.py`), com um

cuidado extra: como o bloco (p, p) de A é identicamente nulo (Seção 2), todo grau de liberdade de pressão livre (sem condição de contorno) tem $D_{ii} = 0$ – nesses casos o pré-condicionador simplesmente não escala aquela componente (fator 1), em vez de dividir por zero.

Testamos empiricamente no sistema real (malha do Poiseuille, $n \approx 19,000$) e o Jacobi *piorou* a convergência em relação a não usar pré-condicionador algum, depois de aplicado o reescalonamento da Seção 6 (900 iterações sem Jacobi contra 1328 com Jacobi para a mesma tolerância). Isso não chega a ser surpreendente: pré-condicionadores diagonais são conhecidos por serem insuficientes para sistemas de sela em geral (a literatura de CFD usa tipicamente pré-condicionadores de bloco, baseados no complemento de Schur do sistema); aqui, uma vez resolvido o problema de escala entre blocos, o ganho adicional do Jacobi não se pagou. Por isso o **femns** não usa pré-condicionador na chamada de produção (`solver.time_step`), embora a função continue disponível em `krylov.py` para uso futuro ou outros problemas.

6 O sistema de sela é mal condicionado: reescalonamento de pressão

6.1 O diagnóstico

Ao rodar o BiCGSTAB (sem pré-condicionador e com Jacobi) no sistema real, encontramos um resultado enganoso: o resíduo relativo caía para valores aparentemente razoáveis, mas a solução obtida tinha até 90% de erro relativo frente à solução de referência (o antigo solve denso). Resíduo pequeno não implicar solução correta é a assinatura clássica de um sistema **mal condicionado**: pequenas perturbações no resíduo (inevitáveis em ponto flutuante, ou por convergência prematura) correspondem a grandes perturbações na solução.

Medindo a norma de Frobenius dos blocos de (1) na malha de teste:

$$\|A_{vv}\|_F = \left\| \frac{1}{\Delta t} M + \frac{1}{Re} K \right\|_F \approx 828, \quad \left\| \begin{pmatrix} G_x \\ G_y \end{pmatrix} \right\|_F \approx 0,71, \quad (5)$$

um desbalanço de escala de $\sim 1000\times$ entre o bloco de velocidade e o bloco de acoplamento de pressão – causado essencialmente pelo fator $1/\Delta t$ ($\Delta t = 0,001 \Rightarrow 1/\Delta t = 1000$) que multiplica M no bloco de velocidade, mas não aparece nos blocos de acoplamento G_x, G_y . Um sistema com blocos de magnitude tão díspares tem número de condição muito maior do que qualquer um dos blocos isoladamente sugeriria, e é exatamente esse tipo de desbalanço que um pré-condicionador puramente diagonal (Jacobi) não corrige, porque ele reescala *linha a linha*, sem levar em conta a relação entre blocos.

6.2 A técnica: reescalonamento diagonal (equilibração)

A correção é uma **mudança de variável** simples. Seja $\beta = \|A_{vv}\|_F / \|(G_x; G_y)\|_F$ e defina a matriz de escala

$$S = \text{diag}(I_n, I_n, \beta I_m), \quad (6)$$

onde $n = n_{\text{pontos}} + n_{\text{elem}}$ (graus de liberdade de velocidade, incluindo o nó de bolha do elemento MINI) e $m = n_{\text{pontos}}$ (graus de liberdade de pressão). Como A é simétrica (Seção 2), aplicamos um reescalonamento **simétrico**, resolvendo o sistema equivalente

$$\tilde{A} \tilde{x} = \tilde{b}, \quad \tilde{A} = SAS, \quad \tilde{x} = S^{-1}x, \quad \tilde{b} = Sb, \quad (7)$$

que tem exatamente a mesma solução física x (basta recuperar $x = S\tilde{x}$ ao final), mas com blocos mais equilibrados:

$$\tilde{A} = \begin{pmatrix} A_{vv} & 0 & -\beta G_x \\ 0 & A_{vv} & -\beta G_y \\ \beta D_x & \beta D_y & 0 \end{pmatrix}, \quad \tilde{x} = \begin{pmatrix} v_x \\ v_y \\ \tilde{p} \end{pmatrix}, \quad \tilde{p} := p/\beta. \quad (8)$$

Ou seja: resolvemos o sistema para uma **pressão escalada** $\tilde{p} = p/\beta$, e recuperamos a pressão física $p = \beta \tilde{p}$ ao final de cada passo de tempo. Os blocos de acoplamento $(-\beta G_x, -\beta G_y, \beta D_x, \beta D_y)$ ficam multiplicados por β , trazendo sua magnitude para perto da do bloco de velocidade A_{vv} (que não é alterado, pois o bloco (p, p) é nulo e $\beta^2 \cdot 0 = 0$, então não há inconsistência ali).

Condições de contorno de pressão. Linhas de A correspondentes a nós com condição de Dirichlet de pressão (ex.: $p = 0$ na saída do Poiseuille) são sobrescritas por `apply_boundary_conditions` depois da montagem de $\tilde{A}$, virando uma linha identidade $1 \cdot \tilde{p}_i = \text{alvo}$. Como essa linha não passa pelo fator β do bloco de continuidade, o alvo correto para impor $p_i = p_{cc,i}$ fisicamente é $\text{alvo} = p_{cc,i}/\beta$ – é exatamente o que `solver.time_step` faz (`b_inf[p_cc_pts] = p_cc[p_cc_pts] / beta`).

6.3 Resultado

Repetindo o mesmo experimento com o sistema reescalado $\tilde{A}$ (sem nenhum pré-condicionador adicional), o BiCGSTAB convergiu para a tolerância $\text{tol} = 10^{-8}$ em 900 iterações, com erro relativo *nulo* (dentro da precisão de ponto flutuante) frente à solução de referência do solve denso – confirmando que o problema era puramente de condicionamento/escala, não um erro na formulação do sistema ou no BiCGSTAB em si.

7 Warm-start entre passos de tempo

Como o método é iterativo, o chute inicial x_0 afeta diretamente quantas iterações são necessárias: o resíduo inicial é $r_0 = b - Ax_0$, e o número de iterações do BiCGSTAB está relacionado ao grau do polinômio φ_m necessário para que $\varphi_m(A)r_0$ fique pequeno – quanto menor $\|r_0\|$ (isto é, quanto mais perto x_0 já estiver da solução verdadeira), menos iterações tendem a ser necessárias.

Como o `femns` usa Euler implícito com Δt pequeno, a solução de um passo de tempo $x^{(k)}$ é, por construção, próxima da solução do passo seguinte $x^{(k+1)}$ (o lado direito $b^{(k+1)}$ muda pouco em relação a $b^{(k)}$, pois depende de $v_x^{(k)}, v_y^{(k)}$ apenas através do termo advectivo). Por isso, `scripts/run_simulation.py` usa a solução do passo anterior (já com as condições de contorno aplicadas, incluindo a conversão $\tilde{p} = p/\beta$) como `x0` do próximo passo, em vez de recomeçar do vetor nulo a cada vez.

Na prática, isso reduziu o número de iterações de forma consistente ao longo dos passos de tempo (Tabela 1): a primeira solução (chute inicial próximo de zero) precisou de mais iterações que as seguintes, que já partem de um estado bem próximo do novo equilíbrio.

Tabela 1: Iterações do BiCGSTAB com warm-start ao longo de 5 passos de tempo consecutivos (malha do Poiseuille, $n \approx 19,000$).

Passo	Iterações	Resíduo relativo final	Tempo acumulado (s)
1	800	$7,4 \times 10^{-9}$	0,44
2	747	$8,0 \times 10^{-9}$	0,79
3	734	$6,1 \times 10^{-9}$	1,14
4	693	$1,0 \times 10^{-8}$	1,46
5	668	$7,5 \times 10^{-9}$	1,77

8 Resumo das mudanças no código

- **Novo módulo `src/femns/krylov.py`:** implementa `bicgstab` (Seção 5) e `jacobi_preconditioner`, ambos agnósticos de *device* – funcionam sobre qualquer tensor esparsa CSR/COO do Py-

Torch, independente do *backend* (CPU, CUDA, ROCm).

- `src/femns/solver.py`: `build_system_matrix` ganhou o parâmetro `beta` (fator de reescalonamento, Seção 6) e a função nova `pressure_scale_factor`, que calcula β a partir das normas de A_{vv} e $(G_x; G_y)$. `time_step` passou a resolver o sistema via `bicgstab` em vez de `torch.linalg.solve(A.to_dense(), b)`, recebendo/devolvendo o vetor de *warm-start* `x0` entre chamadas.
- `scripts/run_simulation.py`: calcula β uma única vez (a matriz A não muda durante a simulação), mantém `x0` entre iterações do laço de tempo, e reporta `bicg_iters/residual` na barra de progresso para diagnóstico. Uma colisão de nome foi corrigida no processo: a variável `x0` do recurso de “ponto móvel” (coordenada de referência do nó deslocado) foi renomeada para `ponto_x0/ponto_y0`, para não conflitar com o `x0` do *warm-start* do solver.
- `tests/test_krylov.py` (novo): valida o BiCGSTAB contra `torch.linalg.solve` em sistemas pequenos não-simétricos e diagonalmente dominantes, confirma que o *warm-start* reduz iterações, e testa o caso degenerado de lado direito nulo.

9 Validação de desempenho e correção

Tabela 2: Comparação entre as abordagens testadas no sistema real ($n \approx 19,000$, GPU AMD RX 6750 XT via ROCm).

Abordagem	Iterações	Resíduo relativo	Erro vs. solve denso
Solve denso (<code>torch.linalg.solve</code>)	–	–	(referência)
BiCGSTAB sem reescalonamento	5000 (não convergiu)	2,78	1213%
BiCGSTAB + Jacobi, sem reescalonamento	4694 (estagnou)	$4,7 \times 10^{-3}$	91%
BiCGSTAB + reescalonamento	900	$6,7 \times 10^{-9}$	$\approx 0\%$
BiCGSTAB + reescalonamento + Jacobi	1328	$7,0 \times 10^{-9}$	$\approx 0\%$

Com reescalonamento e *warm-start*, o tempo por passo de tempo caiu de $\approx 6,5$ s (solve denso) para $\approx 0,35$ s (BiCGSTAB) na mesma malha e GPU – um ganho de $\sim 18\times$ –, com a simulação completa do escoamento de Poiseuille produzindo posições de nó e campos de velocidade/pressão idênticos aos da versão anterior (validado ponto a ponto na trajetória do nó de teste ao longo de várias iterações).

10 Conclusão e limitações

A troca do solve denso por BiCGSTAB só funcionou depois de identificar e corrigir o mau condicionamento estrutural do sistema de sela – sem o reescalonamento de pressão, nenhuma variante testada (com ou sem Jacobi) convergia para a solução correta, apesar de superficialmente parecer convergir (resíduo pequeno, solução errada). Isso reforça um ponto geral de métodos iterativos: **sempre validar a solução contra uma referência independente**, não só olhar o resíduo reportado pelo próprio método.

Limitações conhecidas desta implementação:

- $\text{tol} = 10^{-8}$ e $\text{max_iter} = 2000$ estão fixos como padrão em `time_step`, não expostos no config `yaml` do projeto;
- não há pré-condicionador de bloco/complemento de Schur, que tende a ser necessário para malhas bem maiores (o reescalonamento escalar aqui resolve o desbalanço de *ordem de*

grandeza entre blocos, mas não a estrutura fina do complemento de Schur $D_x A_{vv}^{-1} G_x + D_y A_{vv}^{-1} G_y$);

- o fator β é recalculado apenas uma vez por simulação (a matriz A é constante ao longo do tempo neste modelo); em uma formulação onde A mudasse a cada passo (malha móvel completa, Reynolds variável, etc.), β precisaria ser recalculado também.